Lecture 4: Traditional Process Models — Waterfall, Incremental, Evolutionary, Concurrent

Vedaj Padman [jp_vedaj @cb.amrita.edu]

4.1 Motivation for Traditional Process Models

As software systems began to grow in size, complexity, and organizational importance, informal and ad hoc development practices proved inadequate. Early software projects frequently suffered from cost overruns, schedule slippages, and products that failed to meet user expectations. These failures highlighted the need for disciplined, repeatable approaches that could impose structure on the inherently complex activity of software development.

Traditional process models emerged as systematic responses to these challenges. They sought to bring predictability, control, and managerial visibility to software projects by defining explicit phases, prescribed activities, and well-defined deliverables. These models are often described as **prescriptive** because they specify how development should proceed, what artifacts should be produced, and when progress should be reviewed. Although later lectures will examine adaptive and agile alternatives, understanding traditional models is essential because they form the conceptual foundation from which modern practices evolved.

4.2 The Waterfall Model

The Waterfall model is one of the earliest and most influential software process models. It was motivated by engineering disciplines such as civil and mechanical engineering, where complex systems are constructed through a sequence of carefully planned stages.

The Waterfall model can be defined as a linear, sequential process in which development flows through a series of distinct phases, each of which must be completed before the next begins. Typical phases include requirements analysis, system design, implementation, testing, deployment, and maintenance.

In this model, each phase produces a set of documented outputs that serve as formal inputs to the subsequent phase. The underlying assumption is that requirements can be fully understood early in the project and that changes will be minimal once development is underway.

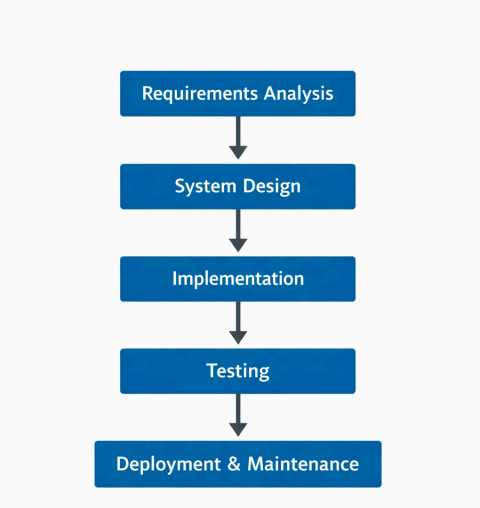


Figure 1: A conceptual representation of the Waterfall model, showing a linear progression through development phases.

The primary strength of the Waterfall model lies in its simplicity and clarity. Clearly defined milestones and documentation-heavy practices make it attractive in environments that demand rigorous oversight, such as defense, aerospace, and regulated industries. However, its rigidity also represents its greatest limitation. When requirements evolve or misunderstandings are discovered late in the lifecycle, accommodating change becomes costly and disruptive.

The implications of the Waterfall model are significant. It emphasizes upfront planning and documentation, prioritizes process control over adaptability, and treats change as an exception rather than a norm. These characteristics make it suitable for stable problem domains but problematic for projects with uncertain or evolving requirements.

4.3 Incremental Process Models

Incremental process models were introduced to address some of the inflexibility inherent in purely linear approaches. The motivation behind incremental development is the recognition that delivering useful software earlier can reduce risk and improve stakeholder confidence.

An incremental model divides the overall system functionality into smaller subsets, known as increments. Each increment is developed using a mini-Waterfall process, resulting in a working piece of software that adds value to the system. Over successive increments, the system grows in capability until the complete product is realized.

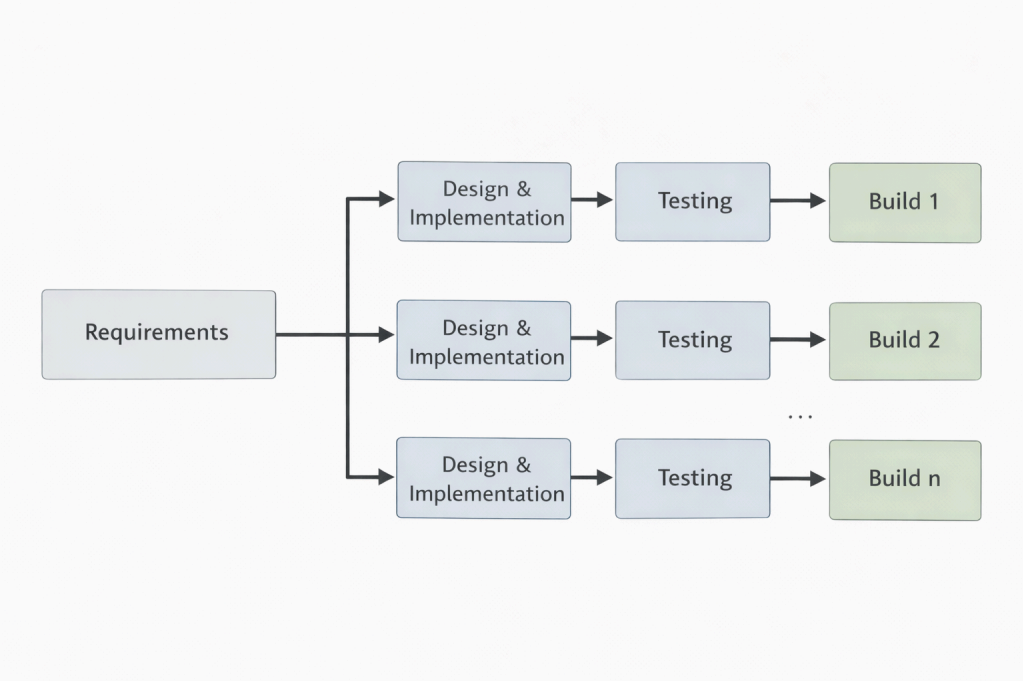


Figure 2: An incremental model illustrating successive delivery of functional system increments.

This approach allows stakeholders to interact with partial implementations early in the development lifecycle. Feedback obtained from early increments can inform subsequent development, thereby reducing the risk of major requirement mismatches.

The implications of incremental models are twofold. On one hand, they improve risk management and provide earlier returns on investment. On the other hand, they require careful planning to ensure that increments are logically structured and that architectural decisions made early do not constrain future extensions. Incremental models thus represent a transition from rigid linearity toward controlled flexibility.

4.4 Evolutionary Process Models

Evolutionary process models arise from the observation that, in many projects, requirements cannot be fully specified at the outset. Instead of treating this uncertainty as a problem, evolutionary models embrace it as an inherent characteristic of software development.

An evolutionary model is defined as a process in which the system is developed through iterative cycles, with each cycle producing a more refined and complete version of the software. Unlike incremental models, where functionality is planned in advance, evolutionary models allow requirements and solutions to co-evolve through continuous feedback.

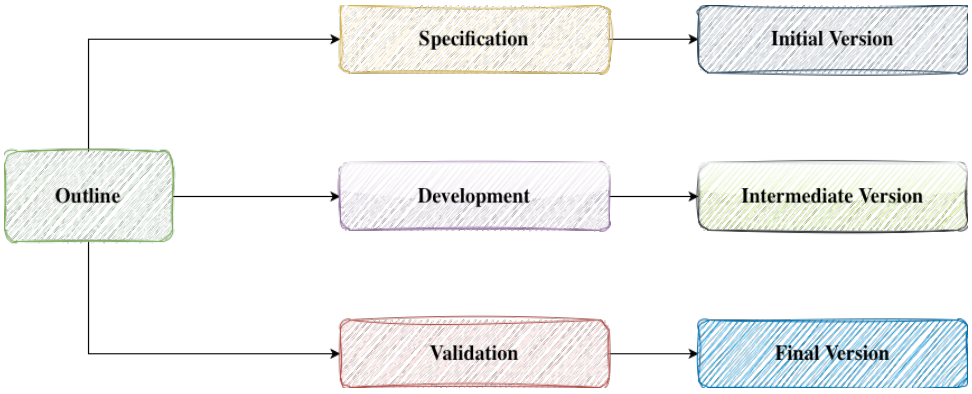


Figure 3: An evolutionary model showing iterative refinement of the software system over multiple cycles.

Common evolutionary approaches include prototyping and the spiral model. Prototyping focuses on building early versions to clarify requirements, while the spiral model explicitly integrates risk analysis into each iteration.

The implications of evolutionary models are profound. They shift emphasis from prediction to learning, from fixed plans to adaptive exploration. While this enhances responsiveness to change, it also demands disciplined management to prevent scope creep and architectural degradation. Evolutionary models thus balance flexibility with the need for ongoing technical and managerial control.

4.5 Concurrent Process Models

Concurrent process models were developed to better reflect the realities of modern software development, where activities often overlap and progress asynchronously. The motivation behind these models is the recognition that software processes are not strictly sequential and that different components may be at different stages of development simultaneously.

A concurrent process model represents development activities as states rather than phases. Each activity—such as requirements analysis, design, or testing—can exist in states such as inactive, under development, under review, or complete. Transitions between states are triggered by events, such as requirement changes or defect discoveries.

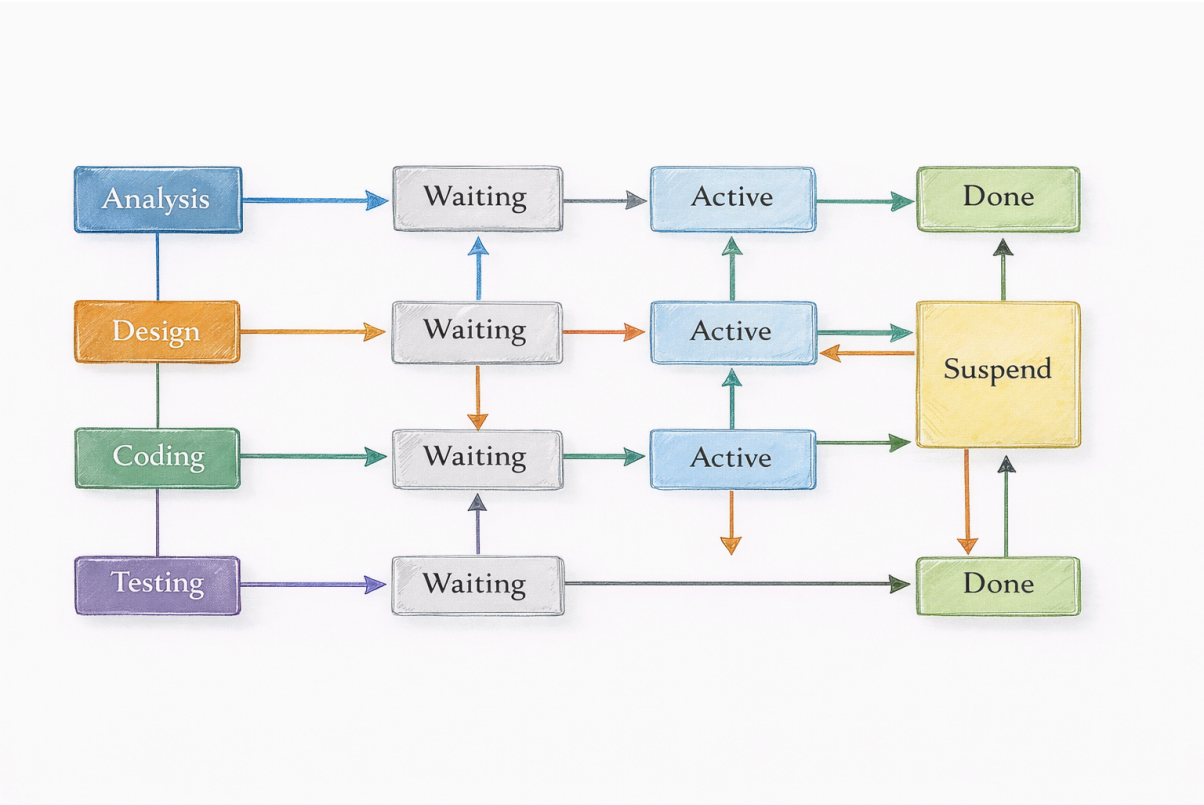


Figure 4: A concurrent process model depicting multiple activities progressing in parallel through different states.

This model provides a more realistic view of software development, especially for large systems with multiple teams and complex dependencies. It supports parallelism and rapid response to change while maintaining explicit visibility into the status of each activity.

The implications of concurrent models include increased managerial insight and improved adaptability. However, they also require sophisticated coordination mechanisms and experienced teams to manage the inherent complexity of parallel activities effectively.

4.6 Comparative Perspective and Conceptual Implications

Traditional process models collectively represent a spectrum of control and flexibility. The Waterfall model emphasizes predictability and order, incremental models introduce staged delivery, evolutionary models prioritize learning and adaptation, and concurrent models reflect the non-linear nature of real-world development.

Understanding these models is critical not because they must be followed rigidly, but because they articulate fundamental trade-offs between planning and responsiveness, control and creativity, and stability and change. These trade-offs will reappear in later discussions of agile and hybrid approaches, which can be viewed as responses to the limitations of traditional models rather than their rejection.

Key Takeaways

- Traditional process models impose structure and discipline on software development.
- The Waterfall model follows a strictly linear sequence of phases with phase completion before progression.
- Incremental models deliver the system in planned functional builds, enabling earlier partial value delivery.
- Evolutionary models develop the system through iterative refinement, allowing requirements and solutions to evolve together.
- Concurrent models represent development as parallel activities progressing through different states rather than a single sequence.
- Early prescriptive models assume relatively stable requirements; volatility increases the cost of late change.
- Models that support iteration or concurrency reduce some change-related risk but require stronger coordination and architectural discipline.
- Selecting a process model reflects trade-offs among predictability, feedback timing, risk exposure, and adaptability.

Key Terminology

- **Waterfall Model**: A linear, sequential software process in which phases proceed in order with formal handoffs.
- **Increment**: A planned, functional build that adds a coherent subset of system capability to prior builds.
- **Incremental Model**: A process in which the product is delivered through multiple increments, each developed via a structured mini-lifecycle.
- **Evolutionary Model**: An iterative process that refines requirements and the solution over successive cycles based on feedback.
- **Prototype**: An early or partial implementation used to explore requirements, reduce uncertainty, or validate feasibility.
- **Spiral Model**: An evolutionary model that structures iterations around explicit risk identification and mitigation.
- **Concurrent Model**: A process model in which multiple activities proceed in parallel, each moving through states such as inactive, active, or complete.
- **Lifecycle Phase**: A major stage of work (such as requirements, design, implementation, testing, or deployment) that produces defined outcomes.
- **Prescriptive Process**: A plan-driven approach that specifies ordered activities, deliverables, and control points to manage development.